**Linux/Unix Operating Systems**, **/proc** is a **virtual (pseudo) file system** used to access **process and kernel information** at runtime.

- /proc is **not stored on disk**.
- It is created **in memory by the kernel**.
- Provides a **file-based interface** to kernel data structures.

# Uses of /proc

## 1. Process Information

Each running process has a directory named by its **PID**.
Contains

- cmdline → command used to start the process
- status → process state, memory usage
- cpuinfo → CPU usage details
- fd/ → file descriptors used by the process

## 2. System Information

- /proc/cpuinfo → CPU details
- /proc/meminfo → Memory usage
- /proc/uptime → System running time
- /proc/loadavg → System load

## 3. Kernel Parameters (Runtime Tuning)

- Used to **read or modify kernel parameters**.
- Example:
- cat /proc/sys/net/ipv4/ip_forward
- Enable IP forwarding:
- echo 1 > /proc/sys/net/ipv4/ip_forward

## 4. Debugging and Monitoring

Tools like top, ps, free, uptime internally **read data from /proc**.

**Why /proc is Important**

- No system calls needed to access kernel info
- Easy **file-based access**
- Useful for **monitoring, debugging, and performance tuning**

**(a) Number of CPU Cores**

**Command:**

# grep -c ^processor /proc/cpuinfo

**Explanation:**

- /proc/cpuinfo lists details of each logical CPU.
- Each CPU entry starts with the line processor : <number>.
- grep -c counts how many lines match processor, i.e., how many cores/logical CPUs are available.

**Sample Output:**

4 - Means the system has **4 CPU cores**.

**(b) Total Memory and Fraction of Free Memory**

**Command:**

## grep -E "MemTotal|MemFree" /proc/meminfo

**Sample Output:**

MemTotal: 8123456 kB

MemFree: 2345678 kB

**To calculate the fraction of free memory:**

awk '/MemTotal/ {total=$2} /MemFree/ {free=$2} END {print "Free Fraction = " free/total}' /proc/meminfo

**Sample Output:**

Free Fraction = 0.2886

About **28.86% of memory** is currently free.

**(c) Number of Processes Currently Running**

**Command:**

ps -e --no-headers | wc -l

or using /proc directly:

ls -d /proc/[0-9]* | wc -l

**Explanation:**

- Each active process has a directory /proc/<PID>.
- Counting these directories gives the total number of active processes.
- ps -e → Displays **all running processes** in the system.
- --no-headers → Removes the header line (like PID TTY TIME CMD) so only process entries remain.
- | (pipe) → Sends the output of ps to the next command.
- wc -l → Counts the **number of lines**.

**Sample Output:**

203

➡ There are **203 processes** currently active.

**(d) Number of Processes in Running and Blocked States**

**Command:**

**grep "procs_running" /proc/stat**

**grep "procs_blocked" /proc/stat**

**Sample Output:**

procs_running 2

procs_blocked 1

**Explanation:**

- procs_running: Number of processes currently **in running state** (ready to run or executing).
- procs_blocked: Number of processes **waiting for I/O** or blocked on resources.

- At this moment, **2 processes** are running and **1 process** is blocked.

**(e) Number of Processes Forked Since Last Boot**

Command:

# grep "processes" /proc/stat

**Sample Output:**

processes 54321

**Explanation:**

- This value represents the **total number of processes created (forked)** since the last boot — including all that have already terminated.

**Comparison:**

The number in /proc/stat (e.g., 54321) is much larger than the currently running processes (e.g., 203 in part c).

That's because **most processes have already exited**, but their creation count remains in this cumulative value.

**(f) Number of Context Switches Performed Since Last Bootup for a Particular Process**

Command:

First, identify a process ID (say bash):

# ps -C bash

Sample output:

```
  PID TTY          TIME CMD
 2310 pts/0    00:00:03 bash
```

Now check its context switch statistics:

cat /proc/2310/status | grep ctxt

**Sample Output:**

voluntary_ctxt_switches: 182

nonvoluntary_ctxt_switches: 37

**Explanation:**

- voluntary_ctxt_switches: Number of times the process **yielded CPU voluntarily** (e.g., waiting for I/O).

- nonvoluntary_ctxt_switches: Number of times the process was **preempted by the scheduler**.

Total context switches = sum of both.